Mini-project: survival predictor

🎯 Today's goal

Today you build one complete machine learning project, from raw table to predictions. You will:

- **clean** the *Northern Star* passenger list from Day 15
- turn text columns into numbers (**features**), including `pd.get_dummies`
- split the data, set a **baseline**, and train logistic regression and a random forest
- **evaluate** them honestly: accuracy, confusion matrix, precision, recall, cross-validation
- **predict** for new made-up passengers, and think about the limits of the model

🤔 Why this matters

Cooking lessons teach you to chop, fry and season. At some point you cook a whole meal, and you find out how the steps fit together. Today is that meal for the Machine Learning phase: every step from Days 16–21 in the right order.

The *Northern Star* is a **made-up** ship. Its passenger list is inspired by the famous Titanic dataset, a classic first project for data scientists, but every passenger and number here is invented by our own code. Because we wrote the code that made the data, we can even check how close our model gets to the "truth".

🚢 Step 1: the passenger list

This is exactly the same code as on Day 15. It makes up 400 passengers with a class (1st, 2nd, 3rd), sex, age, fare, port of embarkation (S, C or Q) and whether they survived. Look at the line with `chance`: in our made-up world, women, 1st class and children had better chances. It also plants some glitches to clean.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass == 2)
+ 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)

ships = pd.DataFrame({
    "passenger_id": np.arange(1, n + 1),
    "pclass": pclass,
    "sex": sex,
    "age": age,
    "fare": fare,
    "embarked": embarked,
    "survived": survived,
})
# Deliberate glitches to clean: 30 missing ages and 3 duplicated rows
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
print("Rows and columns:", ships.shape)
ships.head()
```

```
Rows and columns: (403, 7)
```

	passenger_id	pclass	sex	age	fare	embarked	survived
0	1	3	female	52.0	9.06	S	1
1	2	2	male	48.0	23.24	S	0
2	3	3	male	33.0	10.42	C	1
3	4	3	female	48.0	16.79	S	0
4	5	1	male	30.0	82.88	Q	0

A quick check, as in every EDA (Day 14): what is missing, what is duplicated, and how common is survival?

```
print("Missing values per column:")
print(ships.isna().sum())
print("Duplicated rows:", ships.duplicated().sum())
print(f"Survival rate: {ships['survived'].mean():.2f}")
ships.groupby("sex")["survived"].mean().round(2)
```

```
Missing values per column:
passenger_id     0
pclass           0
sex              0
age             30
fare             0
embarked         0
survived         0
dtype: int64
Duplicated rows: 3
Survival rate: 0.41
sex
female    0.65
male      0.24
Name: survived, dtype: float64
```

🧹 Step 2: clean

Two glitches: 3 duplicated rows and 30 missing ages. Models can't handle missing values, so we fill each one with the **median** age (Day 12). The median isn't pulled around by a few very old or very young passengers.

```
ships = ships.drop_duplicates()
median_age = ships["age"].median()
ships["age"] = ships["age"].fillna(median_age)

print("Rows now:", len(ships))
print("Missing ages now:", ships["age"].isna().sum(), f"(filled with {median_age})")
```

```
Rows now: 400
Missing ages now: 0 (filled with 30.0)
```

🔢 Step 3: turn everything into numbers

Models only do maths, so text columns must become numbers. `sex` has just two values, so `female → 1`, `male → 0` works well, using `.map()` with a dict (Day 3) that says which value becomes which.

`embarked` has three ports: S, C and Q. Writing S=0, C=1, Q=2 would be a mistake: the model would think Q is "twice as much" as C, which means nothing. Instead we make one yes/no column per port, called **one-hot encoding** (exactly one of the columns is "hot", 1, in each row). `pd.get_dummies` does it for us:

```
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})

data = pd.get_dummies(ships, columns=["embarked"], dtype=int)
print("Columns now:", list(data.columns))
data[["sex", "embarked_C", "embarked_Q", "embarked_S"]].head()
```

```
Columns now: ['passenger_id', 'pclass', 'sex', 'age', 'fare', 'survived',
'embarked_C', 'embarked_Q', 'embarked_S']
```

	sex	embarked_C	embarked_Q	embarked_S
0	1	0	0	1
1	0	0	0	1
2	0	1	0	0
3	1	0	0	1
4	0	0	1	0

Now we choose `X` and `y`. The label is `survived`. We also drop `passenger_id`: it's just a counter, so it can't explain survival, and a model might "memorise" IDs instead of learning real patterns.

```
from sklearn.model_selection import train_test_split

X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
print("Features:", list(X.columns))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
print("Train rows:", len(X_train), " Test rows:", len(X_test))
print(f"Survival rate in train: {y_train.mean():.2f}, in test: {y_test.mean():.2f}")
```

```
Features: ['pclass', 'sex', 'age', 'fare', 'embarked_C', 'embarked_Q',
'embarked_S']
Train rows: 300  Test rows: 100
Survival rate in train: 0.41, in test: 0.41
```

`stratify=y` keeps the same survival rate in both parts (Day 16).

🤖 Step 4: baseline, then two models

First the **baseline**: a "model" that always predicts the most common answer (did not survive). Any real model must beat it, or it's useless.

```
from sklearn.dummy import DummyClassifier

baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
baseline_acc = baseline.score(X_test, y_test)
print(f"Baseline accuracy: {baseline_acc:.2f}")
```

```
Baseline accuracy: 0.59
```

Now logistic regression with scaling (Day 18). After training, its `coef_` (coefficients) show the direction of each feature: **positive** pushes towards "survived", **negative** pushes against it. Because the features are scaled, the sizes can be compared.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logreg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
logreg.fit(X_train, y_train)
print(f"Logistic regression test accuracy: {logreg.score(X_test, y_test):.2f}")

coefficients = pd.Series(logreg[-1].coef_[0], index=X.columns)
coefficients.sort_values().round(2)
```

```
Logistic regression test accuracy: 0.63
pclass       -0.52
age          -0.16
embarked_Q   -0.04
embarked_C   -0.04
fare         -0.01
embarked_S    0.06
sex           1.01
dtype: float64
```

`logreg[-1]` is the last step of the pipeline, the LogisticRegression itself.

The model rediscovered our made-up rules: being female (`sex` = 1) helps a lot, and a higher `pclass` number (3rd class) hurts. Next, a random forest (Day 19). We limit `max_depth=4` so the trees can't memorise the training passengers:

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=100, max_depth=4, random_state=42)
forest.fit(X_train, y_train)
print(f"Forest train accuracy: {forest.score(X_train, y_train):.2f}")
print(f"Forest test accuracy:  {forest.score(X_test, y_test):.2f}")
```

```
Forest train accuracy: 0.80
Forest test accuracy:  0.63
```

🔍 Step 5: evaluate honestly

Both models beat the baseline, but not by a lot on this test set. Before drawing conclusions, let's look deeper (Day 20). "Positive" here means "survived":

```
from sklearn.metrics import confusion_matrix, precision_score, recall_score

for name, model in [("logistic", logreg), ("forest", forest)]:
    pred = model.predict(X_test)
    tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
    p = precision_score(y_test, pred)
    r = recall_score(y_test, pred)
    print(f"{name}: TN={tn} FP={fp} FN={fn} TP={tp} precision={p:.2f} recall={r:.2f}")
```

```
logistic: TN=41 FP=18 FN=19 TP=22 precision=0.55 recall=0.54
forest: TN=40 FP=19 FN=18 TP=23 precision=0.55 recall=0.56
```

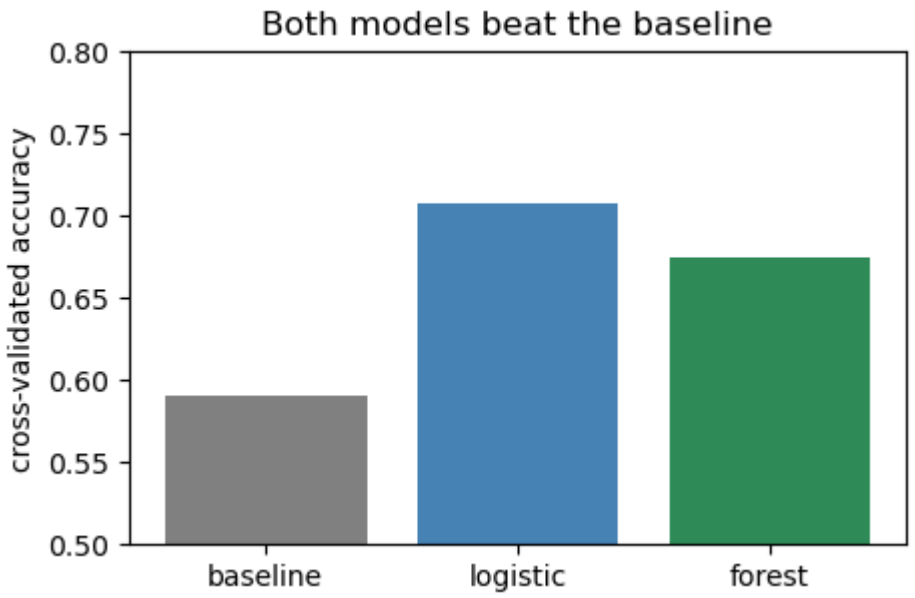
Neither model is a great survivor-finder: recall shows they catch only about half of the survivors, and about half of their "survived" guesses are right (precision). Also, a test set of 100 passengers is small, so one lucky or unlucky split changes the score a lot. Cross-validation (Day 20) gives a steadier comparison of all three:

```
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt

candidates = {"baseline": baseline, "logistic": logreg, "forest": forest}
cv_means = {}
for name, model in candidates.items():
    cv_means[name] = cross_val_score(model, X, y, cv=5).mean()
    print(f"{name:9} mean accuracy over 5 folds: {cv_means[name]:.3f}")

plt.figure(figsize=(5, 3.2))
colors = ["gray", "steelblue", "seagreen"]
plt.bar(list(cv_means.keys()), list(cv_means.values()), color=colors)
plt.ylim(0.5, 0.8)
plt.ylabel("cross-validated accuracy")
plt.title("Both models beat the baseline")
plt.show()
```

```
baseline  mean accuracy over 5 folds: 0.590
logistic  mean accuracy over 5 folds: 0.708
forest    mean accuracy over 5 folds: 0.675
```



🎲 Why isn't it 95%?

Look again at the generator: survival was decided by `rng.random(n) < chance`, a dice roll. A woman in 1st class had a high chance, not a certain one. No model can predict dice. Because we made the data, we can calculate the **best possible** accuracy: always guess the more likely outcome for each passenger, using the true `chance` values from Step 1:

```
true_chance = chance.clip(0, 0.95)          # the real chances from our generator
best_possible = np.maximum(true_chance, 1 - true_chance).mean()
print(f"Best possible accuracy, even knowing the true chances: {best_possible:.2f}")
```

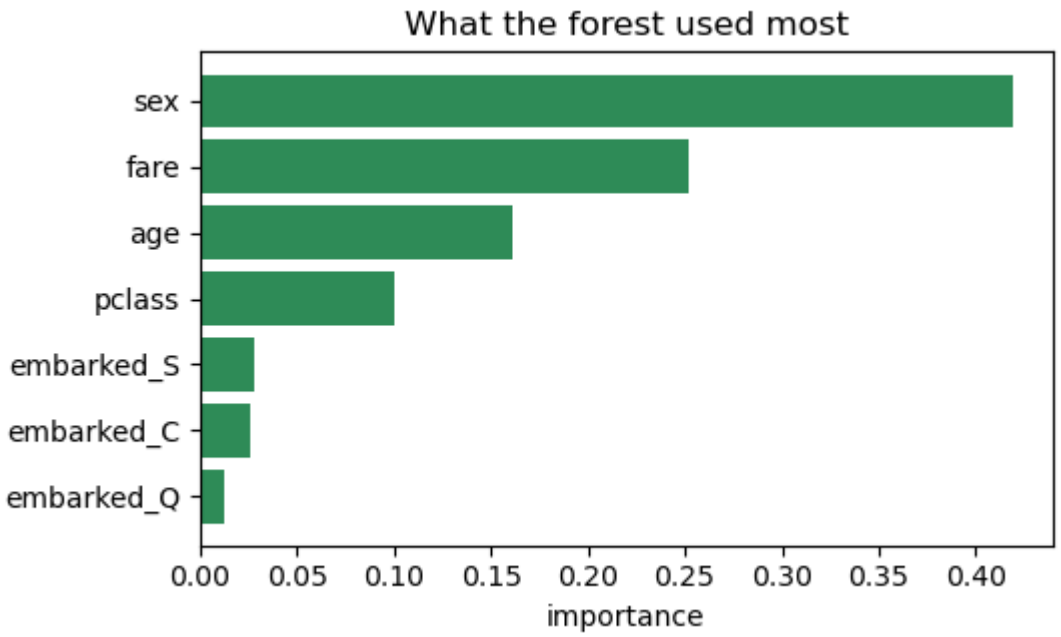
```
Best possible accuracy, even knowing the true chances: 0.74
```

So about 0.74 is the ceiling, and our models reach around 0.7 in cross-validation: close to perfect *for this problem*. Real data is like this too: there is always some randomness no model can explain. A "low" score isn't always a bad model.

Which features did the forest rely on?

```
importance = pd.Series(forest.feature_importances_, index=X.columns).sort_values()

plt.figure(figsize=(5.5, 3.2))
plt.barh(importance.index, importance.values, color="seagreen")
plt.xlabel("importance")
plt.title("What the forest used most")
plt.show()
```



`sex` comes first, as in our generator. `fare` ranks high too: it is strongly linked to class (1st class tickets cost more), so the two share the credit. Remember from Day 19: importance shows what the model *used*, not what causes survival.

🔮 Step 6: predict for new passengers

Three new **made-up** passengers want to know their chances. A tempting first try: put their details in a table, the way a person would write them:

💡 This example has a mistake on purpose. Run it and read the error.

```
new_passengers = pd.DataFrame({
    "pclass": [1, 3, 2],
    "sex": ["female", "male", "male"],
    "age": [8, 30, 40],
    "fare": [90.0, 8.0, 20.0],
    "embarked": ["C", "S", "Q"],
})
logreg.predict_proba(new_passengers)
```

> ⚠️ **ValueError**
>
> The kind of value is right but the value itself isn't, like int("hello").
>
> ```
> Traceback (most recent call last):
>   line 8, in <module>
>     logreg.predict_proba(new_passengers)
>     ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^
> ValueError: The feature names should match those that were passed during fit.
> Feature names unseen at fit time:
> - embarked
> Feature names seen at fit time, yet now missing:
> - embarked_C
> - embarked_Q
> - embarked_S
> ```

The model complains that the columns don't match: it was trained on `embarked_C`, `embarked_Q`, `embarked_S` and numbers for `sex`. **New data must go through exactly the same preparation as the training data.** Let's write the steps once, as a function:

```
def prepare(table):
    """Turn raw passenger rows into the same columns the models were trained on."""
    table = table.copy()
    table["sex"] = table["sex"].map({"male": 0, "female": 1})
    table = pd.get_dummies(table, columns=["embarked"], dtype=int)
    return table.reindex(columns=X.columns, fill_value=0)

ready = prepare(new_passengers)
print(ready)

ready["logistic"] = logreg.predict_proba(ready[X.columns])[:, 1].round(2)
ready["forest"] = forest.predict_proba(ready[X.columns])[:, 1].round(2)
ready[["pclass", "sex", "age", "logistic", "forest"]]
```

```
   pclass  sex  age  fare  embarked_C  embarked_Q  embarked_S
0       1    1    8  90.0           1           0           0
1       3    0   30   8.0           0           0           1
2       2    0   40  20.0           0           1           0
```

	pclass	sex	age	logistic	forest
0	1	1	8	0.84	0.75
1	3	0	30	0.16	0.17
2	2	0	40	0.19	0.17

`reindex(columns=X.columns, fill_value=0)` puts the columns in the training order and adds any that are missing (with only 3 passengers, maybe nobody boarded at one port) filled with 0. The 8-year-old girl in 1st class gets a high chance; the 30-year-old man in 3rd class a low one, just as our made-up rules said.

⚖️ Limits and fairness

- **Made-up data:** these predictions describe our invented ship, not real people or real ships.
- **Small data:** 400 passengers is tiny. Scores move a lot between splits; cross-validation helps.
- **Luck:** some outcomes are random. The model gives a *probability*, not a promise.
- **Fairness:** the model leans heavily on sex and class. That describes who was helped in the (made-up) past; it is not a rule for how people *should* be treated. When models are used for real decisions (loans, jobs, medicine), using features like sex, or features linked to wealth, can repeat old unfairness. Always ask who could be harmed by a model's mistakes.

⚠️ Common mistakes

- Leaving missing values or text columns in `X`. Clean and encode first.
- Encoding categories like ports as 0, 1, 2. Use one-hot columns (`pd.get_dummies`) instead.
- Keeping ID columns or the label itself in `X`.
- Skipping the baseline: 60% accuracy sounds fine until you see that "always no" gets 59%.
- Preparing new data differently from training data → column errors or silent wrong answers.
- Trusting one small test set. Confirm with cross-validation.

✅ Recap: the full recipe

1. **Clean:** `drop_duplicates()`, `fillna(median)`.
2. **Features:** `.map({...})` for two values, `pd.get_dummies` for categories; drop IDs.
3. **Split:** `train_test_split(..., stratify=y, random_state=42)`.
4. **Baseline:** `DummyClassifier(strategy="most_frequent")`.
5. **Models:** scaled logistic regression, random forest with limited depth.
6. **Evaluate:** confusion matrix, precision, recall, `cross_val_score`.
7. **Predict:** prepare new rows the same way, then `predict_proba`.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Each starting box begins with the passenger generator from Step 1; your task is at the end of the box. Solutions are in the answer key at the end.

1. Prepare the features

Starting from the raw `ships` table:

1. remove the duplicated rows and fill the missing ages with the median age
2. turn `sex` into numbers: `male → 0`, `female → 1`
3. one-hot encode `embarked` with `pd.get_dummies(..., dtype=int)`
4. store the features in `X` (without `passenger_id` and `survived`) and the label in `y`

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

# 1. Clean  2. sex to numbers  3. one-hot embarked  4. X and y
```

Hints:

1. `ships = ships.drop_duplicates()` and `ships["age"] = ships["age"].fillna(ships["age"].median())`
2. `ships["sex"] = ships["sex"].map({"male": 0, "female": 1})`, then `data = pd.get_dummies(ships, columns=["embarked"], dtype=int)`
3. `X = data.drop(columns=["passenger_id", "survived"])` and `y = data["survived"]`

2. A fair split

The starting code prepares `X` and `y`. Split them into `X_train`, `X_test`, `y_train`, `y_test` with **25%** for testing, `random_state=42`, and the same survival rate in both parts.

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

# Preparation (the same steps as exercise 1)
ships = ships.drop_duplicates()
ships["age"] = ships["age"].fillna(ships["age"].median())
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(ships, columns=["embarked"], dtype=int)
X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
from sklearn.model_selection import train_test_split

# Split into X_train, X_test, y_train, y_test
```

Hints:

1. `train_test_split(X, y, ...)` gives back four things: `X_train, X_test, y_train, y_test`.
2. 25% for testing is `test_size=0.25`.
3. "The same survival rate in both parts" is what `stratify=y` does.

3. Beat the baseline

Using the split from the starting code:

- store the test accuracy of `DummyClassifier(strategy="most_frequent")` in `baseline_acc`
- train a pipeline `model` of `StandardScaler()` and `LogisticRegression(max_iter=1000)`
- store its test accuracy in `model_acc`. It must be higher than `baseline_acc`.

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

# Preparation (the same steps as exercise 1)
ships = ships.drop_duplicates()
ships["age"] = ships["age"].fillna(ships["age"].median())
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(ships, columns=["embarked"], dtype=int)
X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

```
# Calculate baseline_acc, then train model and calculate model_acc
```

Hints:

1. `baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)`, then `baseline_acc = baseline.score(X_test, y_test)`
2. `model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))`
3. `model.fit(X_train, y_train)` and `model_acc = model.score(X_test, y_test)`

4. What does the forest use?

Train a `RandomForestClassifier(n_estimators=100, max_depth=4, random_state=42)` called `forest` on the training data. Then make a pandas Series `importances` of its feature importances (column names as the index), sorted from biggest to smallest, and store the names of the **two** most important features as a list in `top_two`.

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

# Preparation (the same steps as exercise 1)
ships = ships.drop_duplicates()
ships["age"] = ships["age"].fillna(ships["age"].median())
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(ships, columns=["embarked"], dtype=int)
X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# Train forest, then build importances and top_two
```

Hints:

1. Create and train: `forest = RandomForestClassifier(n_estimators=100, max_depth=4, random_state=42)`, then `forest.fit(X_train, y_train)`.
2. `importances = pd.Series(forest.feature_importances_, index=X.columns).sort_values(ascending=False)`
3. The first two labels: `top_two = list(importances.index[:2])`

5. Challenge: a survival calculator

Finish the function `survival_chance(pclass, sex, age, fare, embarked)`. It gets one passenger's raw details, for example `(1, "female", 8, 90.0, "C")`, and must **return** the survival probability (a number between 0 and 1) from the trained `model`.

Tip: build a one-row DataFrame with exactly the 7 columns of `X`, in the same order, prepared the same way as the training data.

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

# Preparation (the same steps as exercise 1)
ships = ships.drop_duplicates()
ships["age"] = ships["age"].fillna(ships["age"].median())
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(ships, columns=["embarked"], dtype=int)
X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X, y)
print("Columns the model expects:", list(X.columns))

def survival_chance(pclass, sex, age, fare, embarked):
```

```
    # Build a one-row table like X, then return the probability of survival
    return 0.5

print(survival_chance(1, "female", 8, 90.0, "C"))
print(survival_chance(3, "male", 30, 8.0, "S"))
```

Hints:

1. Make a dict with the 7 columns, for example `"sex": 1 if sex == "female" else 0` and `"embarked_C": 1 if embarked == "C" else 0` (the same for Q and S).
2. Turn it into a table with one row: `row = pd.DataFrame([values])[list(X.columns)]`. The `[list(X.columns)]` part puts the columns in the right order.
3. Return the probability of class 1: `return model.predict_proba(row)[0, 1]`

➡️ Tomorrow

Deep learning begins: a neuron from scratch. You've used ready-made models from scikit-learn. Tomorrow you'll build the smallest piece of a neural network yourself with NumPy: inputs times weights plus a bias, an activation function, and a neuron that learns the AND and OR rules on its own.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Prepare the features

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

ships = ships.drop_duplicates()
ships["age"] = ships["age"].fillna(ships["age"].median())
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(ships, columns=["embarked"], dtype=int)

X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
print(X.shape, list(X.columns))
```

2. A fair split

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

# Preparation (the same steps as exercise 1)
ships = ships.drop_duplicates()
ships["age"] = ships["age"].fillna(ships["age"].median())
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(ships, columns=["embarked"], dtype=int)
X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
print(len(X_train), len(X_test), round(y_train.mean(), 2), round(y_test.mean(),
2))
```

3. Beat the baseline

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

# Preparation (the same steps as exercise 1)
ships = ships.drop_duplicates()
ships["age"] = ships["age"].fillna(ships["age"].median())
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(ships, columns=["embarked"], dtype=int)
X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

baseline = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
baseline_acc = baseline.score(X_test, y_test)

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
model_acc = model.score(X_test, y_test)
print(f"Baseline: {baseline_acc:.2f}   Logistic regression: {model_acc:.2f}")
```

4. What does the forest use?

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

# Preparation (the same steps as exercise 1)
ships = ships.drop_duplicates()
ships["age"] = ships["age"].fillna(ships["age"].median())
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(ships, columns=["embarked"], dtype=int)
X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

forest = RandomForestClassifier(n_estimators=100, max_depth=4, random_state=42)
forest.fit(X_train, y_train)
importances = pd.Series(forest.feature_importances_, index=X.columns)
importances = importances.sort_values(ascending=False)
top_two = list(importances.index[:2])
print(importances.round(3))
print("Top two:", top_two)
```

5. Challenge: a survival calculator

```
import numpy as np
import pandas as pd

# --- The Northern Star data (made up), same as in the lesson ---
rng = np.random.default_rng(42)
n = 400
pclass = rng.choice([1, 2, 3], size=n, p=[0.25, 0.25, 0.5])
sex = rng.choice(["female", "male"], size=n, p=[0.4, 0.6])
age = np.round(rng.normal(30, 13, size=n).clip(1, 75))
fare = np.where(pclass == 1, rng.normal(85, 25, n),
       np.where(pclass == 2, rng.normal(25, 8, n), rng.normal(10, 4, n)))
fare = np.round(fare.clip(4, None), 2)
embarked = rng.choice(["S", "C", "Q"], size=n, p=[0.7, 0.2, 0.1])
chance = 0.12 + 0.45 * (sex == "female") + 0.2 * (pclass == 1) + 0.08 * (pclass
== 2) + 0.15 * (age < 12)
survived = (rng.random(n) < chance.clip(0, 0.95)).astype(int)
ships = pd.DataFrame({"passenger_id": np.arange(1, n + 1), "pclass": pclass,
                      "sex": sex, "age": age, "fare": fare,
                      "embarked": embarked, "survived": survived})
missing = rng.choice(n, size=30, replace=False)
ships.loc[missing, "age"] = np.nan
ships = pd.concat([ships, ships.iloc[[5, 17, 42]]], ignore_index=True)
# --- End of the data ---

# Preparation (the same steps as exercise 1)
ships = ships.drop_duplicates()
ships["age"] = ships["age"].fillna(ships["age"].median())
ships["sex"] = ships["sex"].map({"male": 0, "female": 1})
data = pd.get_dummies(ships, columns=["embarked"], dtype=int)
X = data.drop(columns=["passenger_id", "survived"])
y = data["survived"]
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X, y)
print("Columns the model expects:", list(X.columns))

def survival_chance(pclass, sex, age, fare, embarked):
    values = {
        "pclass": pclass,
        "sex": 1 if sex == "female" else 0,
        "age": age,
        "fare": fare,
        "embarked_C": 1 if embarked == "C" else 0,
        "embarked_Q": 1 if embarked == "Q" else 0,
        "embarked_S": 1 if embarked == "S" else 0,
    }
```

```
    row = pd.DataFrame([values])[list(X.columns)]
    return model.predict_proba(row)[0, 1]

print(survival_chance(1, "female", 8, 90.0, "C"))
print(survival_chance(3, "male", 30, 8.0, "S"))
```